Hackathon Project Phases Template that ensures students can complete it efficiently while covering all six phases. The template is structured to capture essential information without being time-consuming.

Hackathon Project Phases Template

Project Title:

Audio transcription app using open ai whisper

Team Name:

Alexa

Team Members:

- Pravallika
- Ruksana
- Ravali
- Charitha
- Vennela

Phase-1: Brainstorming & Ideation

Objective:

The objective of the audio transcription app using OpenAI Whisper is to provide **highly accurate, multilingual, and real-time transcription** of spoken content into text. By leveraging Whisper's advanced speech recognition capabilities, the app aims to overcome challenges such as **background noise, diverse accents, and speaker differentiation**, making transcription more accessible and efficient. It is designed to serve various industries, including media, healthcare, legal, and education, by offering **automated, cost-effective, and scalable transcription solutions** for recorded and live audio.

- Define the purpose and impact of the project.

  The purpose of this project is to develop an AI-powered audio transcription app using OpenAI Whisper to provide **highly accurate, multilingual, and real-time speech-to-**

text conversion. It aims to enhance accessibility, streamline workflows, and reduce transcription costs for industries like media, healthcare, and education. The impact of this solution includes **improved efficiency, better accessibility for the hearing-impaired, and seamless multilingual support**, making transcription faster, more reliable, and scalable for various users.

Key Points:

1. **Problem Statement:** (What problem are you solving?)

The project solves the problem of **inaccurate, time-consuming, and costly transcription** by providing an **AI-powered, highly accurate, and multilingual** speech-to-text solution using OpenAI Whisper.

2. **Proposed Solution:** (Briefly explain your idea)
   The project solves the problem of **inaccurate, time-consuming, and costly transcription** by providing an **AI-powered, highly accurate, and multilingual** speech-to-text solution using OpenAI Whisper. Traditional transcription methods struggle with **background noise, diverse accents, multiple speakers, and language limitations**, leading to errors and inefficiencies. This app addresses these challenges by offering **real-time and batch transcription, speaker differentiation, and noise-handling capabilities**, making high-quality transcription more accessible, automated, and cost-effective for industries like media, healthcare, legal, and education

3. **Target Users:**
   Media & Content Creators
   Business Professionals
   Legal & Healthcare Professionals
   Students & Educators

4. **Expected Outcome:**
   The expected outcome is a **highly accurate, multilingual, and efficient AI-powered transcription app** that converts speech to text in real-time or batch mode. It will enhance **accessibility, save time, and improve workflow efficiency** for various industries.

Phase-2: Requirement Analysis

Objective:

- Technical requirements:

Backend Processing, Multi-Language Support, **Speaker Identification, Noise Reduction, Real-time & Batch Processing, File Support, Security & Privacy.**

- **Functional requirements:**

  Audio files, transcription process, Language support, Speaker identification,etc…

Key Points:

1. **Technical Requirements:** Language support, Audio files, Speaker identification,etc..
2. **Functional Requirements: Real-time transcription, Automatic Language detection, Accurate transcription**
3. **Constraints & Challenges:** Data privacy and security risks, Dependency on openAi's API, User experience and trust, etc..

Phase-3: Project Design

Objective:

- Create the architecture and user flow.

Key Points:

1. **System Architecture Diagram:**

- **Home Screen:**
  - Upload audio file or start live recording
  - Option to choose language or auto-detect
- **Processing Screen:**
  - Displays real-time transcription progress
  - Shows speaker differentiation (if enabled)
- **Transcription Screen:**

- Editable text output
- Speaker labels, timestamps, and highlighting
- Download options (TXT, DOCX, SRT, PDF)

- **Settings & Dashboard:**
  - Manage past transcriptions
  - Select languages, enable noise reduction
  - API integration settings

2. Backend Design:

- **Audio Processing Module:**
  - Accepts audio input (MP3, WAV, FLAC, etc.)
  - Preprocessing (noise reduction, format conversion)
- **AI Model (Whisper) Integration:**
  - Speech-to-text conversion
  - Speaker identification and language detection
- **Post-processing & Storage:**
  - Formats transcription, adds timestamps
  - Saves results to the database
- **User Management & Security:**
  - Secure login, data encryption
  - Cloud storage for transcriptions

2. **User Flow:**

1 **User Sign-up/Login**
- The user creates an account or logs in.
- Option to use third-party authentication (Google, Microsoft, etc.).

2 **Upload or Record Audio**
- User uploads an audio file (MP3, WAV, FLAC, etc.) OR records live audio.
- Option to select language or enable auto-detection.

3 **Processing & Transcription**
- The app processes the file (noise reduction, format validation).
- OpenAI Whisper transcribes the speech into text in real-time or batch mode.

4 **Review & Edit Transcription**

- The user can view the transcription with timestamps and speaker labels.
- Manual editing is allowed for corrections or formatting.

5 **Download & Share**
- The user can export the transcription in different formats (TXT, DOCX, SRT, PDF).
- Option to share via email or third-party integrations (Google Drive, Dropbox).

6 **Dashboard & History**
- Users can access previous transcriptions.
- Option to delete, organize, or reprocess files.

7 **Settings & Preferences**
- Adjust language settings, speaker recognition, and noise filtering.
- Manage API integrations (if needed).
-

3. **UI/UX Considerations:**

    - Simple and intuitive interface
    - Audio file upload recording
    - Progress indicators and real time feed back
    - Transcription accuracy and editing
    - Search and navigation

Phase-4: Project Planning (Agile Methodologies)

Objective:

- Break down the tasks using Agile methodologies.

Key Points:

1. **Sprint Planning:**

    Task1 :- Project planning and research

    Task2 :- UI/UX design and backend development.

    Task3 :- Audio Processing & Storage, Whisper Model Integration.

    Task4 :- User Features & Functionality, Dashboard & History Management.

    Task5 :- Security & Performance Optimization, Testing & Debugging, Deployment & Maintenance

2. **Task Allocation:**

    Team member1 :-task5
    Team member2 :-task4
    Team member3 :-task1
    Team member4 :-task2
    Team member5 :-task3

3. **Timeline & Milestones:**

   Task1:- 4-6 weeks
   Task2:- 4-6 weeks
   Task3:- 4-5 weeks
   Task4:- 4 weeks
   Task5:-4-6 weeks

Phase-5: Project Development

Objective:

- Code the project and integrate components.

Key Points:

1. **Technology Stack Used:**

Frontend:

- **Framework:** React.js / Next.js / Vue.js
- **UI Library:** Tailwind CSS / Material UI
- **File Handling:** FilePond / HTML5 File API

2. Backend (Server & API)

- **Framework:** FastAPI (Python) / Node.js (Express)
- **API Integration:** OpenAI Whisper API
- **Task Queue:** Celery + Redis (for async processing)
- **Authentication:** Firebase Auth / JWT

3. Database & Storage

- **Database:** PostgreSQL / MongoDB
- **File Storage:** AWS S3 / Google Cloud Storage

4. DevOps & Deployment

- **Cloud Provider:** AWS / GCP / Azure
- **Containerization:** Docker + Kubernetes
- **CI/CD:** GitHub Actions / Jenkins

5. Testing & Quality Assurance

- **Unit Testing:** Pytest / Jest
- **UI Testing:** Cypress / Selenium

2. **Development Process:**

Planning & Requirement Analysis

- Define project scope, features, and tech stack.

Backend Development

- Set up API integration with OpenAI Whisper.

Frontend Development

- Build the UI using React/Next.js.
- Implement file upload, playback, and transcription display

Deployment & DevOps

- Deploy on AWS/GCP/Azure using Docker/Kubernetes.

Testing & Optimization

- Conduct unit, integration, and UI testing.
- Optimize performance and security.
  Finalization & Launch
- Gather user feedback, fix bugs, and optimize UX.
- Deploy final version and monitor post-launch performance.

3. **Challenges & Fixes:**

High Latency in Transcription

- **Fix**: Use **GPU acceleration**, optimize **audio preprocessing** .

Handling Large Audio Files

- **Fix**: Implement **chunking** and **streaming transcription**.

Concurrency & Task Management

Fix: Use **Celery + Redis** for async processing and **load balancing** .

Storage & Scalability Issues

Fix: Use **AWS S3 / GCP Storage** for efficient file handling and **CDN caching** for faster access.

API Rate Limits & Costs

Fix: Implement **caching** for repeated requests and optimize API calls with batching.

Security & Authentication

Fix: Use **JWT/Firebase Auth**, encrypt stored files, and apply **role-based access control (RBAC)**.

User Experience & Real-Time Feedback

Fix: Show **progress indicators**, enable **live status updates**, and provide **editable transcripts**.

Phase-6: Functional & Performance Testing

Objective:

- Ensure the project works as expected.

Key Points:

1. **Test Cases Executed:**

OpenAI's Whisper model is a powerful speech-to-text system capable of transcribing audio across various languages and accents. Test cases for Whisper include evaluating basic transcription accuracy, handling multiple speakers, and ensuring it performs well in noisy environments or with low-quality audio. It also tests Whisper's ability to transcribe non-English languages, technical jargon, and long-duration recordings. While the model performs well under most conditions, accuracy may drop with heavy background noise, distorted audio, or complex speech. Overall, Whisper provides robust transcription capabilities, though it is sensitive to input quality.

2. **Bug Fixes & Improvements:**
   Common bugs in an **Audio Transcription App using OpenAI API** include **incomplete transcription output**, which can be fixed by ensuring proper **audio format conversion** before processing. **Slow transcription speed** can be improved with **GPU acceleration**, **multi-threading**, and **asynchronous task queues** like Celery. **Large file upload failures** can be addressed with **chunked uploads** and increased server timeout settings. UI freezing during transcription can be resolved by implementing **WebSockets** or **polling** for real-time updates. Poor transcription accuracy for noisy audio can be improved with **noise reduction** and using **Whisper's larger models**. API rate limits can be managed by **caching repeated transcriptions** and optimizing API call frequency. Lastly, security vulnerabilities in file handling can be mitigated by using **secure cloud storage (AWS S3, GCP)**, validating file uploads, and scanning for potential threats. Implementing these fixes will enhance the app's performance, scalability, and user experience.

   Final Validation:

   - **Functional Testing** – Verify **accurate transcriptions**, smooth file uploads, and API responses.
   - **Performance Testing** – Check **transcription speed**, **large file handling**, and **concurrent requests**.
   - **Security Testing** – Ensure **file encryption**, **authentication (JWT/Firebase)**, and **API rate limits**.
   - **UI/UX Testing** – Test for **responsive design**, **real-time progress updates**, and **usability**.
   - **Edge Case Handling** – Validate **noisy audio**, **low-quality files**, and **various accents/languages**.
   - **Deployment Readiness** – Confirm **CI/CD pipelines**, **logging (Prometheus/Grafana)**, and **scalability**.
3. **Deployment (if applicable):**
   1. Set Up Your Cloud Server
   2. Install Dependencies
   3. Deploy Backend
   4. Deploy Frontend
   5. Set Up Storage & Security
   6. Containerize with Docker

Final Submission

1. **Project Report Based on the templates**
2. **Demo Video (3-5 Minutes)**
3. **GitHub/Code Repository Link**
4. **Presentation**
